# Model Context Protocol (MCP) — Complete Revision Notes

## 1. Definition & Purpose

**Model Context Protocol (MCP)** is an open, standardized protocol that enables seamless, secure connectivity between AI models (clients/hosts) and external data sources, services, and local/remote tools.

- **Standardized Connectivity:** Replaces custom, fragmented N×M integrations between AI models and services with a single universal protocol standard.
- **Context Expansion:** Allows AI systems to query contextual data and execute functions safely across external ecosystems.
- **Modularity & Interoperability:** Decouples model providers from service implementations, allowing independent scaling and tool reuse.

## 2. Architecture & Key Components

MCP uses a client-server architecture where host applications connect to specialized MCP servers to access tools, files, and capabilities.

- **MCP Host / Client:** The application or runtime (e.g., Claude Desktop, IDEs like Cursor/VS Code, AI CLI agents) that initiates requests and coordinates interaction with AI models.
- **MCP Server:** A lightweight backend service or process that exposes specific capabilities (Resources, Tools, Prompts) via standardized MCP endpoints.
- **AI Models:** The LLM (e.g., GPT, Claude, Gemini) that processes context, decides tool executions, and generates user responses.
- **Protocol Layer:** Handles request/response messaging, session management, and capability negotiation (typically over JSON-RPC via stdio or HTTP/SSE).

## 3. Core Primitives

MCP defines four primary building blocks that servers expose to clients:

- **Tools (Functions):** Executable routines or APIs that allow the model to perform side-effects or retrieve dynamic data (e.g., executing code, database queries, API requests).
- **Resources (Context Data):** Static or dynamic contextual information exposed as readable payloads or file structures (e.g., source code fields, database logs, pictures, document files).
- **Prompts (Template Instructions):** Reusable, structured prompt templates and workflow recipes that guide models on how to execute tasks efficiently.
- **Sampling (LLM Inter-Querying):** Allows an MCP server to request LLM completions back through the client host, enabling multi-agent or multi-model workflows without sharing private API keys.

# 4. Workflow & Communication Protocol

- **Handshake & Negotiation:** Client connects to MCP Server and negotiates supported capabilities, protocol versions, and security bounds.
- **Discovery:** Client fetches the list of available Resources, Tools, and Prompts provided by one or multiple connected MCP Servers.
- **Context Injection:** Relevant resources and user instructions are bundled into the prompt context for the primary LLM.
- **Tool Call Execution:** When the model decides to invoke a tool, the client validates the call, sends a request to the appropriate MCP Server, receives the result, and returns it to the model for final synthesis.

# 5. Benefits & Use Cases

- **Unified Standard:** Eliminates the need to write custom integrations for every combination of LLM and external API.
- **Multi-Server Collaboration:** Multiple specialized MCP servers (e.g., GitHub server, Slack server, Postgres server) can run concurrently to fulfill complex workflows.
- **Secure Isolation:** MCP servers run with controlled access boundaries, protecting sensitive data and local environments.

**Key Use Cases:** Developer tools (IDE integrations), database/CRM access, multi-agent AI pipelines, local file system management, and third-party SaaS automation.

# 6. MCP Servers — Comprehensive Notes

## 6.1 What is an MCP Server?

A small program you write that exposes specific capabilities (tools, data, files) in a standard format. Any MCP-compatible AI client (Claude, Cursor, etc.) can discover and use them. The AI never gets direct access to your database or files—your server sits in between and only exposes the functions you choose.

**What a server can expose:**

- **Tools:** Functions the model can call (e.g., *query_orders(customer_id)*).
- **Resources:** Read-only data the model can pull in (files, DB rows, docs).
- **Prompts:** Reusable prompt templates.

## 6.2 Simple Analogy: The Restaurant

- **AI:** Customer
- **MCP Server:** Waiter with a menu
- **Functions / Database / Files:** Kitchen

The customer cannot enter the kitchen. They order from the menu, and the waiter brings back the result. If you didn't write a *delete_everything()* function, the AI cannot execute it.

## 6.3 How It Works (Step-by-Step)

1. Write a server with some functions (e.g., *get_user(email)*, *list_orders()*).
2. Give each function a name and a short description so the AI knows what it does.
3. Connect the server to an AI app (Claude, Cursor, etc.).
4. Ask the AI something (e.g., "show me orders for saksham@gmail.com").
5. The AI reads available functions, picks *list_orders*, and asks your server to run it.
6. Your server runs the real code, talks to the DB, and returns the result.
7. The AI uses the result to answer.

**Minimal Python Example (FastMCP):**

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("my-server")

@mcp.tool()
def get_user(email: str) -> str:
    """Find a user by email"""
    # your real DB query here
    return "Saksham, DTU, 2nd year"

mcp.run()
```

## 6.4 Security & Design Points

- **Direct Access Protection:** The model never connects to Postgres directly. The server runs SQL with its own credentials and returns only the result.
- **Narrow Scope:** Avoid exposing a raw *run_sql(query)* tool unless fully trusted. Prefer narrow tools (e.g., *get_orders_by_user*, *list_products*) or a read-only DB user.
- **Credential Safety:** Credentials stay in the server, not in the model's context.
- **Middleware Logic:** Auth, logging, and rate limits can be added at the server layer.
- **Standardization:** Why MCP instead of custom function calls? It is a standard protocol—write the server once, use it across many clients.

## 6.5 Use Existing Servers or Write Your Own?

**Use existing MCP servers when:**

- You want standard services: GitHub, Notion, Gmail, Google Drive, Postgres, Slack, filesystem, etc.
- You just want to get work done quickly.

**Write your own when:**

- **Custom Logic:** Your data or business logic is custom (e.g., "get my fleet's delayed trucks").
- **Control:** You want fine-grained control over AI actions (e.g., 3 narrow, safe functions).
- **Security:** Third-party servers run code locally and touch credentials. Writing your own ensures code transparency.

- **Quality:** Community servers vary in quality and maintenance.
- **Product Features:** You are building a product and want to expose your app's capabilities.
- **Learning:** You want to understand tool calling deeply (a great afternoon project for GitHub/LinkedIn).

**Rule of Thumb:** Existing ones for common tools, your own for project-specific needs or learning.

## 6.6 Logistics Project Example

**Server 1: Freight** (Custom, wraps DB / FastAPI logic)

- *get_active_shipments()*
- *get_shipment_details(shipment_id)*
- *get_delayed_shipments()*
- *get_route(shipment_id)*

**Server 2: Weather** (Existing or OpenWeather wrapper)

- *get_forecast(city)*
- *get_weather_alerts(region)*

**Unified Prompt Example:***"Which shipments are at risk of delay in the next 24 hours because of weather?"*

**Execution Flow:**

1. Call *get_active_shipments()* on Server 1.
2. Extract route cities for each active shipment.
3. Call *get_forecast(city)* on Server 2 for each location.
4. Synthesize output: "Shipment #42 passes through a storm zone, likely 6h delay."

**Practical Tips:**

- **Server Granularity:** Two servers are modular and reusable (weather works across projects). One server is simpler initially.
- **Thin Wrappers:** MCP tools should wrap existing code rather than reimplementing business logic.
- **Read-Only First:** Start with read operations before adding action capabilities (e.g., *reschedule_shipment()*).
- **Clear Descriptions:** Detailed docstrings (e.g., "Get all shipments currently in transit with their route and ETA") drastically improve tool selection accuracy over simple names like "get shipments".

## 6.7 The Big Idea: Simple Calls, Complex Work

**Developer's Role:** Write small, focused tools that execute single capabilities.
**AI's Role:** Determine tool selection, execution sequence, input parameters, and output synthesis.
**Works Best When:**

- Tools are small, single-purpose, and clearly named.
- Descriptions explicitly state expected return values and use cases.
- Outputs are concise and focused rather than huge data dumps.

**Potential Pitfalls:**

- AI selection errors: wrong tool choice, bad arguments, or omitted steps require verification for critical tasks.
- Vague or overlapping tool names confuse selection models.
- Excessive tools (e.g., >40) degrade reasoning accuracy; keep servers focused.
- Latency & Token Cost: Every tool invocation causes a network round trip and context expansion.

**Architecture Rule of Thumb:**

- **Fixed, Deterministic Logic:** Use standard code/scripts (e.g., daily scheduled reports).
- **Open-Ended Queries:** Use AI + MCP tools for dynamic, unpredictable requests.

## 6.8 Next Steps

1. Build a small Freight MCP server wrapping a Postgres table (FastAPI + Postgres).
2. Add a Weather MCP server (existing or custom wrapper).
3. Connect both to Claude and evaluate the weather-risk query.

Publish the project to GitHub and share on LinkedIn.